

Ex1 (Spanning trees)

Say and prove whether the following problems are in P or are NP-~~complete~~^{HARD}. Given a graph $G = (V, E)$ together with a set of vertices $S \subseteq V$,

- (a) Find a spanning tree such that its leaf set includes S . $S \subseteq L$
- (b) Find a spanning tree such that the set of leaves is exactly S . $S = L$
- (c) Find a spanning tree such that the set of leaves is a subset of S . $L \subseteq S$

A *spanning tree* is a subgraph of G that (1) it is a tree and (2) its vertex set is V .

A *leaf* is a vertex with degree 1.

You can use the fact that the Hamiltonian path problem, given a graph and two vertices a and b , has G a Hamiltonian path starting at a and ending at b ?, is NP-complete.

A path is Hamiltonian if it is a simple path (no vertex is repeated) and it passes through all the vertices in the graph.

a) Aquest opartat pertany a P, si el conjunt S pertany al conjunt de fulles, si eliminem els vèrtexs de S hauríem de seguir tenint una component connexa.

Algorisme

- 1- Creem un G' eliminant tots els vèrtexs de S . Apliquem un DFS, si G' no és connex retornem False. Si no seguim.
- 2- Comprovar que cada $s \in S$ originalment estaven connectats a G' . Si no és així retornem False.
- 3- Com es compleixen totes les condicions es podrà crear un spanning tree amb Prim on $S \subseteq L$.

Creem G' : fer el DFS té cost $O(|V| + |E|)$, i fer Prim té cost $O(|E| \log |V|)$, per tant, l'algorisme és polinòmic.

b) Aquest apartat és NP-HARD, i farem una reducció del tipus:

$$\text{CAM-HAM} \leq_p \text{apartat (b)}$$

Don $f(x) = \text{mateix } G$ i $S = \{a, b\}$ (est palíndrom)

$\Rightarrow$ Si existeix un camí Hamiltonian entre a i b a G , llavors hi ha un arbre on les fulles són exactament $S = \{a, b\}$.
Per definició, el camí és un subgraf de G , on els extrems tenen grau 1 i la resta de vèrtexs tenen grau 2.
Així també és la definició d'un arbre amb exactament dos fulles, per tant, hem trobat un arbre d'expansió amb fulles $S = \{a, b\}$.

$\Leftarrow$ Si tenim un spanning tree amb fulles $S = \{a, b\}$ llavors tenim un camí Hamiltonian que comença a a i acaba en b .

Suposant que existeix un arbre d'expansió T a G , amb fulles (grau 1) $S = \{a, b\}$.

Per teoria de gràfs, si un arbre té algun vèrtex amb $k \geq 3$ tindrà al menys k fulles.

Com tenim 2 fulles, el grau màxim d'un vèrtex intern és 2.

Amb aquestes definicions podem deduir que T en veritat també és un camí Hamiltonian (dos vèrtexs grau 1 i la resta grau 2).

c) Aquest apartat també és NP-HARD, i farem una reducció del tipus:

$$\text{CAM-HAM} \leq_p \text{apartat(c)}$$

Don $f(x) = \text{matriu } G$ i $S = \{a, b\}$ (cost polinòmic)

$\Rightarrow$ Si existeix un camí Hamiltonià a G que comenci a a i acabi en b , llavors hi ha un spanning tree amb fulls L tal que $L \subseteq S$.

Com a l'apartat b, el camí Hamiltonià també és un spanning tree, amb fulls $L = \{a, b\}$.

Com $L = S$ és complet que $L \subseteq S$ i hem trobat una solució vàlida per a l'apartat(c).

$\Leftarrow$ Si existeix un spanning tree tal que la fulls L compleixen que $L \subseteq S$, llavors hem trobat un camí Hamiltonià.

Suposem que tenim un arbre d'expansió T , on $L \subseteq S$ i event $S = \{a, b\}$.

Suposem que T té més de dos nodes, l'arbre tindria ≥ 2 fulls. Com $L \subseteq S$ i $S = \{a, b\}$, l'única forma de que es compleixi tot és que $L = S = \{a, b\}$, i com hem vist abans això es redueix a una instància del camí Hamiltonià.

Ex2 (Color)

Consider the problem COLOR:

INPUT: Undirected graph $G = (V, E)$ and $k \in \mathbb{N}$.

Question: Is there an assignment $c : V \rightarrow \{1, \dots, k\}$ such that, for each $(u, v) \in E$, $c(u) \neq c(v)$.

(a) Show that COLOR belongs to NP.

Consider the following construction that transforms an instance of SAT into a graph:

Let ϕ a boolean formula in CNF on variables $x_1, x_2, \dots, x_n$ and clauses $C_1, C_2, \dots, C_m$.

From ϕ we construct the graph $f(\phi) = (V, E)$ as follows,

$$\begin{aligned} V &= \{v_1, \dots, v_n\} \cup \{x_1, \dots, x_n\} \cup \{\bar{x}_1, \dots, \bar{x}_n\} \cup \{c_1, \dots, c_m\} \\ E &= \{\{v_i, v_j\} : \forall i, j, 1 \leq i, j \leq n, \text{ amb } i \neq j\} \\ &\cup \{\{v_i, x_j\}, \{v_i, \bar{x}_j\} : \forall i, j, 1 \leq i, j \leq n \text{ amb } i \neq j\} \\ &\cup \{\{x_i, \bar{x}_i\} : \forall i, 1 \leq i \leq n\} \\ &\cup \{\{x_i, c_l\} : \forall i, 1 \leq i \leq n, \text{ on } x_i \text{ es un literal a } C_l\} \\ &\cup \{\{\bar{x}_i, c_l\} : \forall i, 1 \leq i \leq n, \text{ on } \bar{x}_i \text{ no es un literal a } C_l\} \end{aligned}$$

(b) Draw the graph resulting after applying f to

$$(\bar{x}_1 \vee x_2) \wedge (x_1 \vee x_3) \wedge (x_1 \vee \bar{x}_2 \vee \bar{x}_3)$$

(c) With the help of f , show that $\text{SAT} \leq_m^p k\text{-Color}$.

a) Per demostrar que $\text{COLOR} \in \text{NP}$ cal demostrar que una possible solució (testimoni) es pot verificar en temps polinòmic.

• Testimoni: donat un graf $G = (V, E)$ i un enter k , el testimoni és una assignació de colors a vèrtexs, una funció $c : V \rightarrow \{1, \dots, k\}$

• Verificar: aquest iterem per cada aresta $(v, u) \in E$, comprovant que $c(u) \neq c(v)$.

El verificar només ha de fer $O(|E|)$ iteracions, i cada comprovació triga $O(1)$. Tenim verificar polinòmic i $\text{COLOR} \in \text{NP}$.

b) Segon la funció $f(\phi) = (V, E)$ els vèrtexs
 Seran: $|V| = 3n + n$

• Base = $\{v_1, v_2, v_3\}$

• Lit. Positive = $\{x_1, x_2, x_3\}$

• Lit. Negative = $\{\bar{x}_1, \bar{x}_2, \bar{x}_3\}$

• Clauses = $\{C_1, C_2, C_3\}$

Arestes:

• Base = $\{(v_1, v_2), (v_2, v_3), (v_1, v_3)\}$

• x_1 i $\bar{x}_1$ e connecten amb v_2 i v_3

• x_2 i $\bar{x}_2$ e connecten amb v_1 i v_3

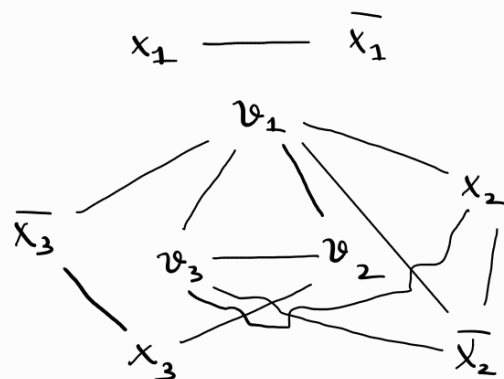
• x_3 i $\bar{x}_3$ e connecten amb v_1 i v_2

• Entre literals = $\{(x_1, \bar{x}_1), (x_2, \bar{x}_2), (x_3, \bar{x}_3)\}$

• C_1 amb $x_1, \bar{x}_2, x_3, \bar{x}_3$

• C_2 amb $\bar{x}_1, x_2, \bar{x}_2, \bar{x}_3$

• C_3 amb $\bar{x}_1, x_2, x_3$



c) La topologia del graf ens obliga a que $k = n+1$.

Formula original satisfactible $\iff$ el graf e $(n+1)$ -colorable.

$\implies$ | Si la fórmula e satisfactible llavors graf e $(n+1)$ -colorable

Sugarem que hi ha una assignació de veritat que satisfi ϕ .

A cada un dels $v_1, \dots, v_n$ li assignem un color diferent, el color $n+1$ l'utilitzarem després.

Si v_i e veritat en la fórmula, assignem el color i al node x_i - i assignem $(n+1)$ a $\bar{x}_i$. Si v_i e mentida, assignem $(n+1)$ a x_i i el color i a $\bar{x}_i$.

Per cada node d'un clàusula C_l : Com la fórmula e satisfactible - hi ha un literal L^* que ho fa certa estant en true. Al estar true ha que tinguí color j - si pertany o veíndole v_j .

Per com hem construït el graf podem assignar a C_l el color j - ja que L^* i v_j mai e falsos. D'aquesta forma tot el graf queda pintat amb $(n+1)$ colors.

$\Leftarrow$ Si el graf $\bar{G}$ és coloreable llavors Φ és satisfactible.

Suposem que el graf té una coloració amb $(n+1)$ colors.

Com abans, cada v_i rep el color i , i el color restant és $(n+1)$.

Tot x_i i $\bar{x}_i$ està connectat a v_j , on $i \neq j$, per tant els colors possibles per a x_i i $\bar{x}_i$ són a $(n+1)$.

Com hi ha una aresta entre ells això obliga a que han de tenir colors diferents.

Definim l'assignació de veritat com: si el node literal té el color i és Verdader. Sinó és Fals.

El node C_i han de tenir un color. No poden tenir $(n+1)$ perquè és connectat amb el complementari del node en la seva clàusula, garantint així que un d'ells té el color $(n+1)$.

Ara, C_i rep el color i , això vol dir que un literal L^* té el color j i és verdader. D'aquesta forma cada clàusula té un color assignat, sent-lo certa, i al final Φ és satisfactible.

Ex3 Geography game.

One way to pass the time during long journey is to play the word game of *Geography*. The first player names a place, say Barcelona. The second player replies with a place whose name starts with the same letter the previous place ended with, such as Athens. The first player could then respond with Sicily, and so on. Each place can only be named once, and the first player who cannot make a move loses.

We can make a mathematical version of this game. We have a directed graph $G = (V, E)$, and a designated start node $s \in V$. The nodes represent the places and the directed edges represent the legal moves. We start at s , and you and I take turns deciding which step to do next. Whoever ends up in a dead end, where every outgoing edge points to a vertex we have already visited, loses. A position or node v is a *winning position* for the first player if whatever legal move the second player selects, the first player never ends up in a dead end.

GEOGRAPHY
→ *Input*: An directed graph $G = (V, E)$ and a start node s .
→ *Question*: Is s a forced win position for the first player? (The first player with strategy s always wins whatever is the strategy selected by the second player).

Show that GEOGRAPHY is in PSPACE (In fact GEOGRAPHY is a well known PSPACEcomplete problem).

Per veure que GEOGRAPHY $\in$ PSPACE cal donar un algorisme que solucioni el problema en espai polinòmic. Aquí l'algorisme:

• Definim 'guanya(v , $unitatz$)', que retorna True si el jugador que que la invoca té una estratègia guanyadora des de v . False d'altra banda.

1- Jugador 1 invoca guanya(s , $\{s\}$)

→ Cas base: Si $\nexists (v, u) \in E$ on ja $u \in unitatz$, llavors retorna False, està en dead end.

→ Cas recursiu: $\exists (v, u) \in E$ tal que u no ha estat visitat.

→ invoquem guanya(u , $unitatz \cup \{u\}$)

- Si tots els n retornen True llavors s'ha de retornar False, ja que el rival pot guanyar amb tota estratègia.
- Si no retorna True, hi ha alguna estratègia que fa perdre al rival.

Al vector vinitatz com a molt pot haver $|V|$ nodes, per tant ocupa espai polinòmic. Vinitatz es pot implementar com a una referència, per lo qual zero no estarà sempre fent còpies.

Per tant, $\text{GEOGRAPHY} \in \text{PSPACE}$.

Ex4 Independent set.

Consider a group of n people. Suppose that the total number of acquaintances (pairs of people which know each other) in the group is $2n$. We model this situation as an undirected graph $G = (V, E)$, in which the vertices are people and we draw an edge between two people if and only if they know each other. The total number of edges is $2n$. In this problem, our aim is to find a fairly large group of people all of whom are strangers. Such a group of people is called an *independent set*.

- (a) We construct a random subset $S \subseteq V$ of people as follows: for each person we independently flip a bias coin with $\Pr[H] = 1/c$, for some constant $c > 1$. If the flip comes up heads, we put the person in S .

What is the expected number of people in S ? (i.e., $\mathbf{E}[|S|]$)

- (b) Let X be a r.v. which counts the number of edges inside S (i.e., edges of the graph that connect pairs of people in S).

What is the value of $\mathbf{E}[X]$?

- (c) Now for each $e \in S$, choose arbitrarily an endpoint of e and remove it.

Find an upper bound to $\mathbf{E}[|S'|]$.

- (d) Argue whether S' is or is not an independent set.

- (e) Using the previous items, give an efficient randomized algorithm that, with probability at least $1/15$, finds an independent set of at least $n/16$ people.

$$a) \quad P(\text{heads}) = \frac{1}{c} \quad X_i = \begin{cases} 0 & \text{si } v_i \text{ no està a } S \\ 1 & \text{si } v_i \text{ està a } S \end{cases}$$
$$X = \sum X_i$$

$$\mathbf{E}[X] = \sum_{\forall v \in V} 1 \cdot \frac{1}{c} + 0 \cdot \left(1 - \frac{1}{c}\right) = \boxed{\frac{n}{c}}$$

$$b) \quad Y_i = \begin{cases} 1 & \text{si } e = (u, v) \text{ està en } S \text{ i tots dos vèrtexs} \\ 0 & \text{d'altra banda} \end{cases}$$

$$P(Y_i = 1) = \text{prob. dos vèrtexs a } S = \frac{1}{c} \cdot \frac{1}{c} = \frac{1}{c^2}$$

$$Y = \sum Y_i \quad \mathbf{E}[Y] = \sum_{e \in E} \frac{1}{c^2} = \boxed{\frac{2n}{c^2}}$$

c) En el cas pitjor tindrem $\frac{2n}{c^2}$ vèrtexs dins de S . Llavors eliminarem $\frac{2n}{c^2}$ vèrtexs. Per tant:

$$|S'| \geq \frac{n}{c} - \frac{2n}{c^2}$$

d) Si que és un independent set. En S podriem pensar que hi haguessin vèrtexs sencera. A S' hem definit interacionalment cada una d'aquelles vèrtexs, generant un independent set.

e) 1 - Creem un S buit

2 - Per cada $v \in V$, tiram una moneda amb probabilitat de head $= \frac{1}{4}$ - si head afegim a S .

3 - Recorrem les arestes de E , si $(u, v) \in E$, tant u i v pertanyen a S , aleatoriament escullim i el eliminem de S .

Per 2 triga $O(|V|)$, i el per 3 triga $O(|E|)$, fent que el cost total sigui $O(|V| + |E|)$

Per maximitzar $|S'| = \frac{n}{c} - \frac{2n}{c^2}$ cal derivar i
igualar a 0: $f(c) = \frac{n}{c} - \frac{2n}{c^2}$

$$f'(c) = n \cdot c^{-2} - 2n c^{-3}, \quad f'(c) = -n c^{-2} + 4n c^{-3}$$

$$0 = -\frac{n}{c^2} + \frac{4n}{c^3}$$

$$\frac{n}{c^2} = \frac{4n}{c^3} \Rightarrow cn = 4n \Rightarrow \boxed{c=4}$$

$$|S'| = \frac{n}{4} - \frac{2n}{16} = \frac{n}{8} \quad \begin{array}{l} \text{(tenir)} \\ n - \frac{7}{8} = \frac{7n}{8} \end{array}$$

Amb Markov i utilitzant el complementari $n - \frac{n}{16} = \frac{15n}{16}$ (valor)

$$P\left(W \geq \frac{15n}{16}\right) \leq \frac{\frac{7n}{8}}{\frac{15n}{16}} = \frac{14}{15}$$

I el seu complementari és $\frac{1}{15}$, just el que volíem.

Ex6 (Pattern matching)

Given a text $T = x_1x_2, \dots, x_n$ and a pattern $S = s_1 \dots s_m$, with $m \ll n$ and both chains over the same alphabet $\Sigma = \{0, 1\}$, we want to determine if S occurs as a contiguous substring of T . For ex., if $T = 10110110010101110$ and $S = 1011$ then 101 101100101011 10 . The standard greedy takes $O(nm)$ steps. There are deterministic algorithms that work in worst-time $O(n + m)$ (Knuth-Morris-Pratt), but they are complicated, difficult to implement and with large implementation constants. The following simple probabilistic algorithm does the job using the fingerprint technique, with a small probability of error. The algorithm computes the fingerprint of S and compares with the fingerprints of successive sliding substring of T , i.e. with $T(j) = x_j \dots x_{j+m-1}$, for $1 \leq j \leq n - m + 1$.

Matching P, T

Express S as an integer $D(S) = \sum_{i=0}^{m-1} x_{i+1}2^i$

so $D(S)$ is a m -bit integer

Choose a prime $p \in [2, \dots, k]$,

where $k = cmn \ln(cmn)$, for suitable $c > 1$

Compute $\phi(S) = D(S) \bmod p$

for $j = 1$ to $n - m + 1$ **do**

 Compute $D(T(j)) = \sum_{i=0}^{m-1} x_{j+i}2^i$

 Compute $\phi(T(j)) = D(T(j)) \bmod p$

if $\phi(T(j)) = \phi(S)$ **then**

output match at position j

endif

endfor

Prove,

- (a) This algorithm is one-side, it may output match when there is no match. Prove that $\Pr[\text{output match, when no match}] \leq 1/c$, for suitable $c > 0$.
- (b) Prove that the algorithm can be implemented in $O(n + m)$ steps.

a) Si hi ha un fals positiu vol dir que $\phi(T(j)) = \phi(S)$
però que $T(j) \neq S$. Llavors: $D(T(j)) \equiv D(S) \bmod p$
Això vol dir que p és divisor de $\Delta = |D(T(j)) - D(S)|$.
La diferència com a molt és 2^m
Com el nombre de divisors primers és el $\log_2 2^m$ hi ha
 m primers 'dolents' a que fan que hi hagi un fals positiu.

Com es veu la finestra de $1 - n - m + 1$ tindrem
com a molt $m + m$ primers diferents.

Escollim p d'un interval $[2, k]$, i el nombre de
primers $\leq k$ és: $\pi(k) \approx \frac{k}{\ln(k)}$

Aplicant $k = cmn$:

$$\pi(k) \approx \frac{cmn \ln(cm n)}{\ln(cm n \ln(cm n))} \approx cm n$$

La probabilitat de que just surti un primer diferent que

beni fals positiu és: $P_{\pi}(\text{fals positiu}) = \frac{\text{tot. prim. tal}}{\text{tot. primer}} = \frac{mn}{cmn} = \frac{1}{c}$

b) Per veure que l'algoritme triga $O(m+n)$ passa, cal veure
que per passar de $D(T[i])$ a $D(T[i+1])$ es pot fer en temps
 $O(1)$. Restant el primer x_j de $D(T[i])$. Dividint tot entre 2
per a desplaçar la resta de x_{j+i} . Sumant al valor $x_{j+m} \cdot 2^{m-1}$ (el
nou caràcter de la finestra).

Llavors, calcular el valor $\phi(S)$ triga $O(m)$ ja que iterem sobre
 m caràcters. Calcular el primer $\phi(T[i])$ també triga $O(m)$.
I calcular els següents té cost $O(1)$ com acabem de veure, i
com fem $(n-m)$ iteracions el cost seria $O(1) \cdot O(n-m) = O(n-m)$.
Cost total = $O(m+m+n-m) = O(m+n)$

Ex6 (Words)

Consider the following problem whose input is a $n \times n$ grid, where each *square* contains a letter. Also, given as a part of the input, there is a database with $m < n^2$ words, all the words have the same length ℓ , so the input's size is $(n^2 + m\ell)$. Each one of the words in the database has been placed in an horizontal or vertical position in the grid (in ℓ consecutive squares, as in the database or in reversed order). In this version of the problem words can't be in diagonal and the intersection of two words is empty. The remaining positions of the square have been filled arbitrary with letters.

The example below has $n = 8, w = 7, l = 4$ with data base (VUIT, HASH, ROCA, TRES, TRET, CLAU, KILO).

L	A	S	B	A	R	C	A
A	A	F	D	H	S	A	H
R	O	C	A	Z	U	C	E
I	T	T	I	U	V	U	K
S	I	L	A	L	B	A	I
A	C	O	B	U	K	L	L
L	B	T	E	R	T	C	O
B	E	L	T	R	E	S	Z

Design an expected linear time randomized algorithm to find the positions in the grid of all the mots from the database. For each word, the algorithm must return the coordinates of the square, where it starts. Justify the time complexity of your algorithm as a function of n, m, ℓ .

Per resoldre el problema utilitzarem una taula de Hash i la tècnica de Rolling Hash. La taula de Hash tindrà mida m .

Algorisme

- 1 - Creem una taula de Hash T buida, amb una funció h que permeti la comparació entre paraules diferents. (Simple uniforme, amb $w = \Theta(m)$)
- 2 - Per cada paraula x , fem $T(x)$ i $T(x\text{-revertida})$.
- 3 - Creem $pos = \{\}$
- 4 - Per tota fila de la graella, ogem una finestra de ℓ posicions
 - Si la finestra actual no retorna coincidència a T llavors no és una de les restes paraules.
 - Si la finestra actual retorna una coincidència, hem trobat

no de les restes paraula. $pos = pos \cup \{fil\}$
→ Mover la finestra i tornar a mirar, així fins acabar la fila.

5- Repetir el mateix procés, però ara per a les columnes.

6- Retornem pos.

Anàlisi del cost

- Fer les insercions té cost $O(1)$, i fem m insercions de paraules de longitud l , així doncs un cost de $O(ml)$.
- Al tenir una funció h simple uniforme, amb $w = \Theta(n)$, les binquetes a T tenen cost $O(1)$. A una fila analitzem n fer n cerques, per tant el cost per fila és $O(n)$. Com hi ha n files el cost serà $O(n^2)$.
- El mateix succeix per a les columnes, un cost $O(n^2)$.

Finalment el cost és lineal: $O(ml + n^2 + n^2) = O(ml + n^2)$, lineal segons l'input.